Robotics Tutorials Packet

TurtleBot 4 (OAK-D Pro) + Duckiebot Labs + Gaussian Splatting Extensions

Prepared for: Introduction to Robotics (Undergraduate)
Date: January 21, 2026
Version: 2.0

Introduction

This packet supports an undergraduate Introduction to Robotics sequence from first power-on to autonomous navigation. Part I provides a TurtleBot 4 (ROS 2) tutorial roadmap that uses the platform's Create 3 base, 2D LiDAR, and OAK-D Pro stereo camera [1]. Part II provides eight beginner-friendly Duckiebot labs built around Duckietown Shell (dts) and a Raspberry Pi 4 base station [2]. Part III adds two extension labs on 3D Gaussian Splatting (3DGS) for creating a photorealistic 3D scene model from captured images [3].

Each lab is designed for students with no prior Linux, ROS, Docker, or embedded hardware experience. Every activity includes (i) a narrative that explains why the step matters, (ii) a checklist-style procedure, and (iii) a results table to record observations. Instructors can run this packet as a linear sequence, or assign individual labs as stand-alone modules.

Learning goals

- Set up a safe development workflow (edit → run → observe → debug) on real robot hardware.
- Understand core robotics message flows: sensor data in, commands out, feedback through logs/visualizations.
- Build perception modules for lane, color, and object detection using standard computer vision tools.
- Implement and evaluate closed-loop control (e.g., PID) for lane following and simple behaviors.
- Use mapping and navigation concepts (SLAM + localization) on a ROS 2 platform (TurtleBot 4).
- Optionally capture and reconstruct a 3D environment model using Gaussian Splatting for visualization and analysis.

How this packet is organized

- Part I — TurtleBot 4 tutorial roadmap: assembly/setup → ROS 2 tools → OAK-D Pro integration → SLAM → Nav2.
- Part II — Duckiebot labs (1–8): beginner setup → image streaming → vision pipelines → lane following control → autonomy.
- Part III — Gaussian Splatting extension labs (9–10): dataset capture → reconstruction → export and evaluation.
- Each lab ends with a results table and a short references list to support further study.

Assumptions

- Students have no prior experience with Linux, ROS, Docker, or embedded hardware.
- Work is performed in teams of 2–4 students with one robot per team.
- The base station (Raspberry Pi 4, 4GB or similar) is used as the student workstation for coding and monitoring.
- Each lab is designed to fit within a 1–3 hour session.

Safety (Read Before Lab Work)

- Keep hands, hair, and loose clothing away from moving wheels.

- Test autonomy at low speed first; always have a quick way to stop the robot (pick up robot, power switch, or emergency stop command).
- Use a clear test area; keep cables off the floor in driving zones.
- If the robot behaves unexpectedly: stop it first, then debug.

Getting help and troubleshooting

- If something fails, stop the robot first, then diagnose using logs and visualization tools (dashboard, RViz2, rqt_image_view).
- Work in pairs: one person drives/monitors safety; one person records results and edits code.
- When you ask for help, include: (i) the command you ran, (ii) the full error message, and (iii) what you expected to happen.
- Duckietown troubleshooting pages and FAQs are often the fastest way to resolve setup issues [4].

References

[1] TurtleBot 4 User Manual. "Features." Accessed January 21, 2026. https://turtlebot.github.io/turtlebot4-user-manual/overview/features.html

[2] Duckietown. "Duckietown Shell (dts) Installation." Duckiebot Operation Manual. Accessed January 21, 2026. https://docs.duckietown.com/daffy/opmanual-duckiebot/setup/setup_laptop/setup_dt_shell.html

[3] Kerbl, Bernhard, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. "3D Gaussian Splatting for Real-Time Radiance Field Rendering." arXiv, 2023. https://arxiv.org/abs/2308.04079

[4] Duckietown. "Duckiebot FAQ Guide." Duckiebot Operation Manual. Accessed January 21, 2026. https://docs.duckietown.com/daffy/opmanual-duckiebot/debugging_and_troubleshooting/faq/index.html

Table of Contents

Introduction 2
Table of Contents 4
Part I — TurtleBot 4 Roadmap (OAK-D Pro) 5
Part II — Duckiebot + Raspberry Pi 4 Labs 7
Lab 1 — Power-On, Network, SSH 8
Lab 2 — Base Station Setup + DTProject 9
Lab 3 — Stream Camera to Base Station 11
Lab 4 — Color Detection (HSV) 13
Lab 5 — Lane Center + Error 15
Lab 6 — Wheel Commands + PID 17
Lab 7 — Object Detection + Streaming 18
Lab 8 — Integrated Autonomy 20
Lab 9 — Capture Dataset for 3DGS 22
Lab 10 — Train/Export 3DGS 24

Part I — TurtleBot 4 Tutorial Roadmap (OAK-D Pro)

This roadmap provides a suggested sequence from setup to autonomous navigation, including OAK-D Pro integration.

Part	Name	Description	Repository	Video
1	Environment setup (PC + ROS 2 + tools)	Install ROS 2 on your development PC, create a workspace, and install TurtleBot 4 desktop tools (RViz2, teleop, etc.).	Link	Link
2	Unboxing & physical setup	Unbox, check components, set up dock/charger, power on, and perform basic safety checks.	Link	Link
3	Robot networking & remote access	Connect via AP mode, configure Wi-Fi, and verify you can SSH into the robot.	Link	Link
4	First ROS 2 node (Python) for TurtleBot 4	Create a ROS 2 Python node and interact with basic topics (cmd_vel, odom) to validate your dev workflow.	Link	Link
5	Simulation workflow (before real robot)	Run TurtleBot 4 in simulation for fast iteration before deploying to hardware.	Link	Link
6	Real robot bringup & sensor sanity check	Launch the robot stack, confirm LiDAR and camera topics, then drive with teleop and visualize in RViz2.	Link	Link
7	OAK-D Pro tooling install (DepthAI)	Install DepthAI and verify the OAK-D Pro enumerates correctly; run a	Link	Link

Part	Name	Description	Repository	Video
		basic camera test.		
8	OAK-D Pro ROS 2 integration (RGB/depth/point cloud)	Run the ROS 2 driver for OAK cameras and visualize RGB/depth/pointcloud topics in RViz2.	Link	Link
9	Mapping (SLAM Toolbox)	Create a 2D occupancy-grid map by driving the robot and save it for navigation.	Link	Link
10	Autonomous navigation (Nav2)	Localize on a saved map and send navigation goals; tune costmaps and recovery behaviors.	Link	Link

Part II — Duckiebot + Raspberry Pi 4 Base Station Lab Manuals

Labs 1–8 build core skills from setup to autonomous navigation with image processing. Labs 9–10 are extensions on 3D Gaussian Splatting for capturing and visualizing a 3D environment.

Lab 1: First Power-On, Network Connection, and SSH

Time: 1–2 hours

Checkoff / deliverable: Demonstrate (a) `dts fleet discover` showing your robot as Ready, (b) a successful `ping`, and (c) a successful SSH login.

Learning objectives

- Power on the Duckiebot and base station safely.
- Confirm the robot is reachable on the network.
- Use Duckietown Shell to discover robots and confirm status.
- SSH into the robot and run a simple command.

Safety notes

- Do not place the robot on the floor until you are ready to test; keep it on a stand or hold it while validating network steps.
- If the robot starts moving unexpectedly, lift it immediately.

Narrative

Robots are computers with motors and sensors. Before we write any autonomy code, we must be able to power the robot safely, connect it to the network, and access it remotely. This lab teaches the minimum skills needed to begin developing and debugging: discovery, ping, and SSH. [1-3]

Experiment setup

Hardware

- Duckiebot (powered and assembled), microSD card inserted, battery charged.
- Raspberry Pi 4 base station (monitor/keyboard/mouse).
- A Wi-Fi network reachable by both the robot and the base station (lab router recommended).

Software (base station)

- Terminal access on the base station.
- Duckietown Shell (`dts`) installed (if not installed, proceed to Lab 2 first).

Procedure

A. Power on and wait for boot

1. Power on the Duckiebot and wait until it completes boot.
2. Place the robot where it has good Wi-Fi signal (near router if possible).

B. Discover the robot on the network

On the base station, open a terminal and run:

```
dts fleet discover
```

You should see your robot hostname and a status indicator. Wait until it shows Ready.

C. Ping and SSH

Once you know the robot hostname, test network reachability:

```
ping <ROBOT_HOSTNAME>.local
```

If ping works, SSH into the robot (default username/password depends on your course image):

```
ssh duckie@<ROBOT_HOSTNAME>.local
```

After logging in, run one harmless command to confirm you are on the robot:

```
hostname
uname -a
```

D. Common beginner troubleshooting

- If `dts fleet discover` shows nothing: confirm the robot and base station are on the same Wi-Fi.
- If `.local` names do not resolve: use the router's device list to find the IP address, then ping/SSH the IP.
- If SSH fails: confirm the correct username and password for your lab image (ask instructor if unsure).

Results table (fill in)

Robot hostname	Ready in `dts fleet discover`? (Y/N)	Ping works? (Y/N)	SSH works? (Y/N)	Notes / troubleshooting

References

[1] Duckietown. "First Boot." The Duckietown Manual (ente). Accessed January 21, 2026. https://docs.duckietown.com/ente/duckietown-manual/10-setup/03-duckiebot/duckiebot-first-boot.html.

[2] Duckietown. "Setup - Booting the Duckiebot." Duckiebot Operation Manual (daffy). Accessed January 21, 2026. https://docs.duckietown.com/daffy/opmanual-duckiebot/setup/setup_boot/index.html.

[3] Duckietown. "Install Duckietown Shell (dts)." Duckiebot Operation Manual (daffy). Accessed January 21, 2026. https://docs.duckietown.com/daffy/opmanual-duckiebot/setup/setup_laptop/setup_dt_shell.html.

Lab 2: Base Station Environment Setup and Your First DTProject

Time: 2–3 hours

Checkoff / deliverable: Run a DTProject on the Duckiebot and show output logs (e.g., a “Hello from robot” message).

Learning objectives

- Install Duckietown Shell (`dts`) on the Raspberry Pi base station.
- Clone a template project (DTProject) and understand the project structure.
- Build and run the project on the Duckiebot.
- Practice the edit → run → observe workflow.

Narrative

Duckietown software is designed to run in containers so that every student team uses the same dependencies and the same runtime environment. In this lab you install the Duckietown Shell (the command-line tool), create a new DTProject from a template, and run it on the robot. [1-4]

Experiment setup

Hardware

- Duckiebot powered on and visible in `dts fleet discover`.
- Raspberry Pi 4 base station on the same network.

Software (base station)

- Git installed.
- Docker installed and running.
- Python environment capable of installing `dts` (pip or pipx).

Procedure

A. Install Duckietown Shell (`dts`)

Follow the Duckietown installation instructions for your OS. On many systems, the recommended install uses pipx.

```
pipx install duckietown-shell
which dts
```

Confirm that `which dts` prints a path.

B. Create a new DTProject from a template

Duckietown provides a basic project template for a minimal DTProject. Create a new repo from the template and clone it to the base station.

```
git clone <YOUR_REPO_URL>
cd <YOUR_REPO_FOLDER>
```

C. Build and run on the Duckiebot

From inside the DTProject folder, build on the robot (the -H flag targets the robot by hostname):

```
dts fleet discover
dts devel build -f -H <ROBOT_HOSTNAME>
dts devel run -H <ROBOT_HOSTNAME>
```

When the container runs, you should see log output in the terminal.

D. Make a tiny edit and re-run

3. Open one source file and change a message text or log rate.
4. Re-run the build and run commands.
5. Verify the output changes as expected.

Results table (fill in)

Base station OS	`which dts` output	DTProject repo name	Build succeeded? (Y/N)	Run succeeded? (Y/N)	Notes / errors

References

[1] Duckietown. "Install Duckietown Shell (dts)." Duckiebot Operation Manual (daffy). Accessed January 21, 2026. https://docs.duckietown.com/daffy/opmanual-duckiebot/setup/setup_laptop/setup_dt_shell.html.

[2] Duckietown. "DTProject." Duckietown Developer Manual (ente). Accessed January 21, 2026. https://docs.duckietown.com/ente/devmanual-software/beginner/dtproject/index.html.

[3] Duckietown. "Create a New DTProject." Duckietown Developer Manual (ente). Accessed January 21, 2026. https://docs.duckietown.com/ente/devmanual-software/beginner/dtproject/create-project.html.

[4] Duckietown. "How to Create a Duckietown Project." Duckietown Developer Manual. Accessed January 21, 2026. https://docs.duckietown.com/ente/duckietown-robotics-development/out/duckietown-developer-manual/dtproject/how-to-create-a-duckietown-project.html.

Lab 3: Camera Streaming to the Raspberry Pi Base Station (3 Methods)

Time: 1–2 hours

Checkoff / deliverable: Show a live camera feed on the base station and capture at least one frame image file.

Learning objectives

- View the Duckiebot camera stream on the base station.
- Capture a frame to disk for later processing.
- Compare at least two viewing methods and note FPS/latency.

Narrative

Before autonomy, you need to see what the robot sees. Duckietown supports multiple ways to view the camera stream, including a browser dashboard and a dedicated image viewer. This lab also introduces beginner habits: verify inputs, record examples, and measure basic performance. [1-4]

Experiment setup

Hardware

- Duckiebot on the network and reachable.
- Raspberry Pi base station with display.

Software

- Duckietown Shell (`dts`) installed.
- Web browser (for dashboard method).

Procedure

Method 1: Duckiebot Dashboard (browser)

Launch the dashboard and open the camera-related page(s):

```
dts duckiebot dashboard <ROBOT_HOSTNAME>
```

Method 2: Duckietown Image Viewer (recommended)

Open the Image Viewer and select the camera topic. Use Space to capture a frame to disk.

```
dts duckiebot image_viewer <ROBOT_HOSTNAME>
```

- Press X to increase FPS, Z to decrease FPS.
- Press Space to capture a frame (save it for Lab 4).
- If the image is black, check for a lens cap or poor lighting.

Method 3: GUI tools + rqt_image_view

Start GUI tools, then run rqt_image_view and select the topic `/ROBOT/camera_node/image/compressed`:

```
dts start_gui_tools <ROBOT_HOSTNAME>
# in the GUI tools terminal:
rqt_image_view
```

Record your approximate FPS and whether it is stable.

Results table (fill in)

Method	Worked? (Y/N)	Approx FPS	Notes (latency, stability, issues)

References

[1] Duckietown. "Image Streaming and Image Viewer." Duckiebot Operation Manual (daffy). Accessed January 21, 2026. https://docs.duckietown.com/daffy/opmanual-duckiebot/operations/03-subsystem-testing/make-it-see-duckiebot-image-streaming.html.

[2] Duckietown. "Duckietown Dashboard (Browser UI)." The Duckietown Manual. Accessed January 21, 2026. https://docs.duckietown.com/ente/duckietown-manual/20-operations/01-duckiebot-handling/duckietown-dashboard-browser-ui.html.

[3] Duckietown. "Make It See (start_gui_tools, rqt_image_view)." Duckiebot Operation Manual (daffy). Accessed January 21, 2026. https://docs.duckietown.com/daffy/opmanual-duckiebot/operations/make_it_see/index.html.

[4] Duckietown. "Camera Reader (topic names and message types)." Duckietown Developer Manual (ente). Accessed January 21, 2026. https://docs.duckietown.com/ente/devmanual-software/beginner/ros/camera-reader.html.

Lab 4: Image Processing I: Color Detection with OpenCV (HSV Thresholding)

Time: 2–3 hours

Checkoff / deliverable: Demonstrate a live mask that reliably highlights the target color under normal lab lighting.

Learning objectives

- Understand why HSV is commonly used for color detection.
- Implement a mask using `cv.inRange()` and clean it using morphology.
- Tune thresholds for at least one lane color and one marker color.
- Overlay detection results for debugging.

Narrative

Color detection is one of the simplest perception tools for robotics. The key idea is to convert the image from BGR/RGB to HSV, then threshold a range of HSV values. You will also learn that raw masks are noisy; morphological operations (erosion/dilation/open/close) help remove speckles and fill holes. [1-4]

Experiment setup

Hardware

- Duckiebot camera stream working (Lab 3).
- A lane segment or colored paper/tape to detect.

Software

- Python with OpenCV available inside your DTProject container or on the base station.
- A saved sample frame from Lab 3 (optional, recommended for offline tuning).

Procedure

A. Start with a saved image (offline tuning)

- Use the Image Viewer to capture one frame (Space key) and save it.
- Open that image in a small OpenCV script to tune thresholds before going live.

B. Convert BGR to HSV and threshold

Core OpenCV steps:

```
hsv = cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, lower_hsv, upper_hsv)
```

Tune lower/upper HSV values until the mask isolates your target color.

C. Clean the mask using morphology

```
kernel = np.ones((5,5), np.uint8)
mask_clean = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
mask_clean = cv2.morphologyEx(mask_clean, cv2.MORPH_CLOSE, kernel)
```

D. Go live

6. Subscribe to the live camera stream.
7. Apply HSV thresholding and morphology per frame.
8. Display an overlay (original image + mask) and verify robustness.

Results table (fill in)

Target color	Lower HSV (H,S,V)	Upper HSV (H,S,V)	Morphology used	Worked live? (Y/N)	Notes

References

[1] OpenCV. "Changing Color Spaces." OpenCV-Python Tutorials. Accessed January 21, 2026. https://docs.opencv.org/4.x/df/d9d/tutorial_py_colorspaces.html.

[2] OpenCV. "Thresholding Operations Using inRange." OpenCV-Python Tutorials. Accessed January 21, 2026. https://docs.opencv.org/4.x/da/d97/tutorial_threshold_inRange.html.

[3] OpenCV. "Morphological Transformations." OpenCV-Python Tutorials. Accessed January 21, 2026. https://docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html.

[4] Duckietown. "Image Streaming and Image Viewer." Duckiebot Operation Manual (daffy). Accessed January 21, 2026. https://docs.duckietown.com/daffy/opmanual-duckiebot/operations/03-subsystem-testing/make-it-see-duckiebot-image-streaming.html.

Lab 5: Lane Perception: Compute Lane Center and Cross-Track Error

Time: 2–3 hours

Checkoff / deliverable: Show an annotated live stream (or logs) with lane center and a numeric error signal updating in real time.

Learning objectives

- Subscribe to the Duckiebot compressed camera topic in a ROS node.
- Decode `sensor_msgs/CompressedImage` into an OpenCV image.
- Compute a lane center estimate in a region of interest (ROI).
- Compute a cross-track error signal for control.

Narrative

Autonomous navigation needs a number that says how far the robot is from the desired path. In Duckietown lane following, that number is often a lateral error between the image center and the lane center. This lab turns your color mask (Lab 4) into a control-ready error signal. [1-3]

Experiment setup

Hardware

- Duckiebot on a lane track or with clear lane-like colored lines in view.

Software

- Your DTProject container environment (Lab 2).
- HSV thresholds from Lab 4.

Procedure

A. Confirm the camera topic name

/<ROBOT_HOSTNAME>/camera_node/image/compressed

B. Subscribe and decode compressed images

- Subscribe to the topic using a ROS subscriber.
- Decode the JPEG/PNG bytes to a NumPy image (OpenCV).
- Verify the image is the expected resolution and orientation.

C. Compute lane center and cross-track error

9. Crop an ROI (bottom 1/3 or bottom 1/2 of the image).
10. Apply your HSV mask (Lab 4) to the ROI.
11. Compute the x-coordinate of the mask centroid (mean x of nonzero pixels).
12. Compute error_px = centroid_x - image_center_x.
13. Normalize error to [-1, 1] if desired.

D. Visual debug

- Draw the ROI, the centroid point, and an arrow showing error direction.
- Stream the annotated image to the base station (or view it in an OpenCV window).

Results table (fill in)

Image size (WxH)	ROI (y-range)	Centered error (px)	Left offset error (px)	Right offset error (px)	Notes

References

[1] Duckietown. "Camera Reader (topic names and message types)." Duckietown Developer Manual (ente). Accessed January 21, 2026. https://docs.duckietown.com/ente/devmanual-software/beginner/ros/camera-reader.html.

[2] ROS 2 Documentation. "sensor_msgs/CompressedImage." Accessed January 21, 2026. https://docs.ros2.org/foxy/api/sensor_msgs/msg/CompressedImage.html.

[3] Duckietown. "Duckiebot Camera Subscriber (example)." The Duckietown Manual. Accessed January 21, 2026. https://docs.duckietown.com/ente/duckietown-manual/70-developer-manual/ros/duckiebot-camera-subscriber.html.

Lab 6: Control: Publish Wheel Commands and PID Lane Following

Time: 2–3 hours

Checkoff / deliverable: Duckiebot follows the lane for at least 10 seconds with stable behavior at low speed.

Learning objectives

- Publish wheel commands to drive the Duckiebot.
- Implement a proportional (P) controller, then add D (and optionally I).
- Tune gains to reduce oscillations and drift.
- Use safe speed limits and timeouts.

Safety notes

- Start at very low speed. Have one person designated as the safety spotter.
- If the robot leaves the lane, pick it up immediately (do not chase it).

Narrative

In this lab you close the loop: perception produces an error signal, and control turns that error into wheel commands. PID control is a widely used feedback controller that uses proportional, integral, and derivative terms based on tracking error. [1-3]

Experiment setup

Hardware

- Duckiebot in a clear lane track area with good lighting.

Software

- Lane error node from Lab 5 producing error in real time.
- A ROS publisher node you will write to publish wheel commands.

Procedure

A. Open-loop wheel command test

- Publish small wheel velocities and confirm forward motion and turning.
- Add a safety timeout: if no new camera frames arrive, command zero speeds.

B. Implement P (then PD/PID) control

```
error = lane_error
P = Kp * error
D = Kd * (error - prev_error) / dt
I = I + Ki * error * dt   # optional
u = P + D + I
```

```
v_left  = base_speed - u
v_right = base_speed + u
```

C. Tuning workflow

14. Set Ki = 0 and Kd = 0. Increase Kp until the robot corrects but oscillates.
15. Increase Kd in small steps until oscillations reduce.
16. Only add Ki if you see consistent drift that never corrects.
17. Record gains and behavior for each trial.

Results table (fill in)

Trial #	base_speed	Kp	Ki	Kd	Behavior	Avg |error| over 10s

References

[1] Duckietown. "Wheel Control (publish wheel commands)." Duckietown Developer Manual (ente). Accessed January 21, 2026. https://docs.duckietown.com/ente/devmanual-software/beginner/ros/wheel-control.html.

[2] Duckietown. "PID Control (slides)." Duckietown Instructor Manual. Accessed January 21, 2026. https://docs.duckietown.com/daffy/instructor-manual/resources/slides/control/03-pid.html.

[3] Duckietown. "PID Control Learning Experience (setup)." The Duckietown Manual. Accessed January 21, 2026. https://docs.duckietown.com/ente/duckietown-manual/60-learning-experiences/lx-setup-pid-control.html.

Lab 7: On-Board Inference: Object Detection and Annotated Streaming

Time: 2–3 hours

Checkoff / deliverable: Demonstrate live detections (bounding boxes) on the base station while inference runs on the robot.

Learning objectives

- Run an object detector on the robot computer (on-board inference).
- Publish detections (class, confidence, bounding boxes).
- Stream an annotated image to the base station for monitoring.

- Measure inference FPS and end-to-end latency.

Safety notes

- Run detection at a low driving speed or while the robot is stationary for initial tests.
- Do not test object-triggered behaviors at high speed.

Narrative

Color thresholding finds simple features; object detection identifies semantic objects (e.g., a duckie, cone, or sign). In this lab, the model runs on the robot (not the base station). The base station acts as a viewer and logger. [1-3]

Experiment setup

Hardware

- Duckiebot camera stream working.
- One or more target objects (duckie, cone, printed marker) placed in view.

Software

- A provided pre-trained model (instructor supplies weights).
- Your camera subscriber pipeline from Lab 5.

Procedure

A. Choose a lightweight detector

- Use a small model to keep FPS reasonable on embedded hardware.
- Start with a high confidence threshold (e.g., 0.5) to reduce false positives.

B. Run inference on the robot

18. Subscribe to the camera topic on the robot.
19. For each frame, run the detector and obtain bounding boxes + labels + confidences.
20. Publish detections on a topic (or print to logs for initial debugging).

C. Stream annotated results to the base station

- Draw boxes/labels on a copy of the frame.
- Publish the annotated image as a compressed image topic.
- View it using the Image Viewer or rqt_image_view.

D. Measure performance

- Record approximate inference FPS.
- Record approximate end-to-end latency (camera to annotated view).

Results table (fill in)

Test condition	Confidence threshold	Detected? (Y/N)	Approx FPS	False positives?	Notes

References

[1] Duckietown. "Object Detection (slides)." Duckietown Instructor Manual. Accessed January 21, 2026. https://docs.duckietown.com/daffy/instructor-manual/resources/slides/perception/04-object-detection.html.

[2] TensorFlow. "TensorFlow Lite Object Detection on Raspberry Pi (example)." GitHub repository. Accessed January 21, 2026. https://github.com/tensorflow/examples/blob/master/lite/examples/object_detection/raspberry_pi/README.md.

[3] Duckietown. "Image Streaming and Image Viewer." Duckiebot Operation Manual (daffy). Accessed January 21, 2026. https://docs.duckietown.com/daffy/opmanual-duckiebot/operations/03-subsystem-testing/make-it-see-duckiebot-image-streaming.html.

Lab 8: Integrated Autonomy: Lane Following + Stop Line + Object-Aware Behavior

Time: 2–3 hours

Checkoff / deliverable: Run 3 successful trials: lane follow, stop at stop line, resume; demonstrate correct response to a detected object.

Learning objectives

- Implement a simple state machine for autonomous driving behaviors.
- Detect a stop line using color segmentation and ROI logic.
- Combine lane following (Lab 6) with object-triggered actions (Lab 7).
- Run repeatable trials and record failures.

Safety notes

- Keep speed low and test each behavior separately before integrating.
- Use a clear lane segment; remove clutter that can confuse vision-based detection.

Narrative

Real autonomy is not a single algorithm; it is a system of behaviors. This lab integrates perception (color + object detection) with control (lane following) using a simple state machine that is easy to debug. [1-4]

Experiment setup

Hardware

- Duckietown track with a clearly visible stop line (colored tape).
- A target object placed in the lane.

Software

- Lane follower node from Lab 6.
- Stop-line HSV thresholds from Lab 4.
- Object detector from Lab 7.

Procedure

A. Define your states

- LANE_FOLLOW
- STOP_AT_LINE
- WAIT (N seconds)
- RESUME

B. Stop line detection rule

21. Choose a bottom ROI where the stop line will appear.
22. Apply your stop-line HSV mask to the ROI.
23. If masked pixels exceed a threshold, trigger STOP_AT_LINE.
24. Add a cooldown timer to prevent retriggering.

C. Object-aware rule

- If object detected in central ROI above threshold, stop or slow. Prefer STOP for safety.

D. Testing protocol

25. Test stop line detection while stationary (move robot by hand).
26. Test lane following alone.
27. Enable stop line behavior and test at low speed.
28. Enable object behavior and test with object placed/removed.
29. Run 3 full trials and record outcomes.

Results table (fill in)

Trial #	Lane follow stable? (Y/N)	Stop line success? (Y/N)	False stop? (Y/N)	Object response correct? (Y/N)	Notes

References

[1] Duckietown. "Lane Following Demonstration." The Duckietown Manual (ente). Accessed January 21, 2026. https://docs.duckietown.com/ente/duckietown-manual/40-demonstrations/lane-following-lf.html.

[2] OpenCV. "Thresholding Operations Using inRange." OpenCV-Python Tutorials. Accessed January 21, 2026. https://docs.opencv.org/4.x/da/d97/tutorial_threshold_inRange.html.

[3] Duckietown. "PID Control (slides)." Duckietown Instructor Manual. Accessed January 21, 2026. https://docs.duckietown.com/daffy/instructor-manual/resources/slides/control/03-pid.html.

[4] Duckietown. "Object Detection (slides)." Duckietown Instructor Manual. Accessed January 21, 2026. https://docs.duckietown.com/daffy/instructor-manual/resources/slides/perception/04-object-detection.html.

Lab 9: Gaussian Splatting I: Capture a Dataset for 3D Environment Reconstruction

Time: 1–3 hours

Checkoff / deliverable: Submit a dataset folder containing extracted frames and estimated camera poses (or COLMAP outputs).

Learning objectives

- Plan a capture path with good overlap and parallax.
- Record a short video of a static scene using an on-board camera (e.g., OAK-D Pro).
- Extract frames and estimate camera poses using an SfM toolchain (COLMAP via Nerfstudio).
- Perform basic dataset sanity checks.

Narrative

3D Gaussian Splatting reconstructs a scene as a set of 3D Gaussians optimized from multi-view images. Reconstruction quality depends heavily on data capture: you need overlapping views and parallax. This lab focuses on capturing and processing data into a format suitable for training a splat model. [1-5]

Experiment setup

Hardware

- Robot with an on-board camera (OAK-D Pro recommended).
- Raspberry Pi 4 base station for recording and file transfer.
- A static capture area with good texture and stable lighting.

Software

- A way to record video from the on-board camera (DepthAI tools if using OAK-D Pro).
- GPU workstation recommended for processing/training.
- Nerfstudio installed on processing machine; COLMAP and FFmpeg installed.

Procedure

A. Plan your capture

- Scene must be mostly static (avoid moving people).
- Move slowly to avoid motion blur.
- Overlap: each view should share features with nearby views.
- Parallax: translate around objects (not only rotate).

B. Record a video

- Record 30–90 seconds at a steady speed.
- Keep exposure stable; avoid sudden turns.

C. Process into a dataset with poses

```
ns-process-data video --data /path/to/video.mp4 --output-dir /path/to/output_dataset
```

- Verify extracted images exist.
- Verify COLMAP outputs/poses exist.
- If processing fails, revisit capture quality (blur/texture/overlap).

Results table (fill in)

Capture device	Video length (s)	Resolution	Approx FPS	Processing succeeded? (Y/N)	Approx # frames	Notes

References

[1] Kerbl, Bernhard, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. "3D Gaussian Splatting for Real-Time Radiance Field Rendering." arXiv, 2023. Accessed January 21, 2026. https://arxiv.org/abs/2308.04079.

[2] Nerfstudio Project. "Custom Dataset Quickstart." Nerfstudio Documentation. Accessed January 21, 2026. https://docs.nerf.studio/quickstart/custom_dataset.html.

[3] Nerfstudio Project. "ns-process-data (CLI Reference)." Nerfstudio Documentation. Accessed January 21, 2026. https://docs.nerf.studio/reference/cli/ns_process_data.html.

[4] COLMAP. "Tutorial." Accessed January 21, 2026. https://colmap.github.io/tutorial.html.

[5] Luxonis. "DepthAI Example: Record Video." DepthAI Documentation. Accessed January 21, 2026. https://docs.luxonis.com/software-v3/depthai/examples/record_replay/record_video/.

Lab 10: Gaussian Splatting II: Train, View, and Export a 3D Gaussian Splat Model

Time: 1–3 hours (training time depends on GPU)

Checkoff / deliverable: Submit screenshots/screen recording and the exported .ply file (or link).

Learning objectives

- Train a Gaussian Splat model using Nerfstudio (Splatfacto).
- Inspect the reconstructed scene in the viewer.
- Export the splat as a .ply file for sharing/viewing.
- Evaluate quality using a simple rubric and note common artifacts.

Narrative

Training converts your captured dataset into a 3D representation. Splatfacto in Nerfstudio provides an accessible pipeline for Gaussian splat training and export. This lab emphasizes practical evaluation: identify artifacts (floaters, holes, ghosting) and connect them back to capture quality. [1-3]

Experiment setup

Hardware

- GPU workstation recommended.

Software

- Nerfstudio installed.
- Dataset processed in Lab 9 (images + poses).

Procedure

A. Start training

```
ns-train splatfacto --data /path/to/output_dataset
```

- If you hit out-of-memory errors, reduce dataset size or use a smaller configuration if available.
- Train until the viewer shows a stable scene.

B. View the reconstruction

- Orbit the scene in the viewer.

- Look for floaters, holes, blur, and ghosting.

C. Export the splat

ns-export gaussian-splat --load-config <path_to_config.yml> --output-dir exports/splat

D. Record results

- Save 3 screenshots or a 10–20 second screen recording.
- Record the exported .ply file size.

Results table (fill in)

Dataset path	GPU model	Viewer loaded? (Y/N)	Main artifact	Overall quality (1–5)	Exported .ply size (MB)	Notes

References

[1] Nerfstudio Project. "Splatfacto." Nerfstudio Documentation. Accessed January 21, 2026. https://docs.nerf.studio/nerfology/methods/splat.html.

[2] Nerfstudio Project. "ns-export gaussian-splat (CLI Reference)." Nerfstudio Documentation. Accessed January 21, 2026. https://docs.nerf.studio/reference/cli/ns_export.html.

[3] Kerbl, Bernhard, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. "3D Gaussian Splatting for Real-Time Radiance Field Rendering." arXiv, 2023. Accessed January 21, 2026. https://arxiv.org/abs/2308.04079.